

OPENCOVEN · COVEN-THREADS V0.2 PHASE 0

The weave *holds* or it doesn't.

An authority-boundary gate layer for agentic memory — so a familiar's identity survives compaction, injection, serialization, and any writer who shouldn't have a pen.

DESIGN FROZEN · 2026-07-14

ENFORCEMENT · PHASE 1+

🌿 Sage · 🎭 Echo · 👑 Nova · ⚡ Cody · ✨ Charm · ❖ Val

The problem

Agentic memory is *untrustworthy* by default.

⚠️ Forced compaction

The runtime evicts context to fit the window. Your SOUL.md summary replaces your SOUL.md source. The familiar drifts.

⚠️ Serialization drop

Export → import round-trips through a format that strips `readOnly`. The familiar is portable, but its protections are not.

⚠️ Prompt injection

A cooperating attacker inside the model's context nudges it into rewriting its own identity file. The familiar becomes someone else.

⚠️ Unauthorized writer

Any process that reaches disk can edit a protected surface. Boundary exists on paper, not in structure.

Four failure modes. One shared property: *enforcement lives somewhere the familiar can reach.*

Where it sits

The stack.

Line one

Gate 4 fail-closed is not *a feature*.

It is a *conformance requirement*. An implementation that allows Gate 4 to be bypassed does not conform to RFC-0001.

— RFC-0001 §5.4 · Nova non-negotiable #2

✓ What this means in practice

- Unknown surface path → **Reject**
- Unknown thread for a protected surface → **Reject**
- Unknown channel → **Reject**
- Validator panic → **Reject with diagnostic**
- Any bypass path in the type system → **compile error**

This is stated at line one of the spec, not saved for Phase 1 hardening. If it isn't the first property, the layer isn't the layer.

The core move

The vocabulary *is* the architecture.

Four words. Each bound at first use to a concrete referent. If the metaphor drifts ahead of the referent, the code stops matching the language, and we become Letta-shaped: beautiful vocabulary, unclear semantics.

Thread

authority relationship: *surface* → *writer*

One thread per `(surface, writer)` pair. Holds under load or snaps.

Strand

fibers inside a thread

Hash · signature · manifest entry · audit trail · serialization marker.

Weave

the enforced pattern of threads

The invariant that these specific threads must all hold together for identity to be coherent.

Channel

axis of load the thread must survive

Deliberate · Forced · Serialization · Mutation.

Metaphor named by ❖ Val (2026-07-14). Rebound to concrete referents by 🌌 Echo (v0.1.1 substrate-authority pass).

Thread

A thread has *tension*.

A **Thread** is a directional line from a protected surface to the writer authorized to modify it. It's not a contract — that's too static. It's a load-bearing connection with three states:

Holds

All strands intact under the channel of load.

Frayed

A strand failed. The thread hasn't snapped.
Must surface to the operator.

Snapped

The thread failed. The surface goes read-only until repair.

Every gate check reduces to one question:

Does thread **T** hold under channel **C** ?

Weave

Ward's gates are the *loom*. Not the threads.

A **Weave** is the invariant that a specific set of threads must all hold together for a familiar's identity to be coherent.

Ward v0.2's four gates are the structure the weave is made *on*. Threads run through them. The gates give the weave its shape; they aren't threads themselves.

A weave is **coherent** iff its pattern predicate holds. When a thread snaps, the weave surfaces *which* surface degraded — not just "something is wrong."

Corrected by 🌀 Echo (v0.1.1) after the v0.1 sketch loaded threads-as-static-contracts and gates-as-threads onto the wrong axis.

Coherence states

- **Coherent** — pattern predicate holds
- **Degraded** — specific surfaces named
- **Broken** — pattern violated

Anti-pattern

Enforce on the *predicate*. Not the descriptor.

⚠ Descriptor-vs-predicate drift

A weave's pattern is defined by a **predicate** — a function that returns *coherent* / *degraded* / *broken*. The predicate carries a derived **descriptor** for humans, tools, and Cave rendering.

If anything downstream ever *gates enforcement on the descriptor instead of the predicate*, we have reinvented the derived-index problem one layer up.

✓ The rule

Enforcement lives on the authoritative object.

Legibility lives on the derived one.

Named in prose by 🌺 Echo (v0.1.1 third turn). Same source-authoritative discipline the memory retrieval substrate resolved — applied one layer up.

Strand

Threads *fray* before they snap.

A **Strand** is a single fiber inside a thread. Multi-strand threads survive stress. A thread survives a channel iff *all its strands* survive that channel.

ContentHash

The bytes match. Cheapest strand, strongest under Forced compaction.

Signature

A named key signed this. Attributable, revocable.

ManifestEntry

Listed in an external manifest that survives the harness restarting.

AuditTrail

First seen, event log reference. Chain of custody.

SerializationMarker

Format version + contract hash. Explicit round-trip survival contract. Load-bearing for C7.

"Thread frayed at strand `ContentHash` — SOUL.md hash mismatch, detected on channel `Forced` ."

Channel

Threads hold under *specific* load.

Deliberate

Promotion, dreaming, memory flush. *Familiar-initiated, principal-gated.*

Serialization

Export / import round-trip. *Format-mediated.*

Threads here must carry a `SerializationMarker`. This is the channel `C7` governs.

Forced

Context auto-compact. *Runtime-initiated, no familiar cooperation available.*

Strands here must survive without agent-side intervention. This is the channel `WARD-C1-C6` governs.

Mutation

Direct client mutation via the daemon. *Default channel.*

Every thread holds under this or the daemon rejects at the boundary.

The v0.1.1 correction: threads don't just "hold." They hold *under channels*. Load has an axis, and the axis is first-class in the type system.

The invariants

Five channel-survival requirements.

1 · Identity-as-memory-property

Protected surfaces are typed memory layers, not runtime config. Threads bind to typed surfaces at construction.

2 · Structural mutation authority

The gate is external. Cannot be cooperated past. Enforcement is Rust-side, called by the daemon — not by the familiar.

3 · Two-compaction contract — inherited

`Deliberate` and `Forced` are distinct channels with distinct thread-survival requirements. **WARD-C1-C6** = "these are the threads that must hold under `Forced`."

Canonical home: `coven-grimoire` Ward Layer Spec Brief §9. Inherited by reference; not reinvented here.

4 · Survives serialization — new, numbered C7

Every thread that must round-trip carries a `SerializationMarker` strand. Export → import produces a weave with equivalent tension state, or fails visibly.

Numbered C7 so lineage from C1-C6 is preserved. Not orphaned. Not tacked-on.

None of the four ships alone. Phase 0's design must show how all four are co-designed in the type system — concretely, via the `Channel` enum.

The load-bearing artifact

The Channel enum.

```
pub enum Channel {  
    Deliberate,    // promotion, dreaming, flush; familiar-initiated  
    Forced,        // context auto-compact; runtime-initiated  
    Serialization, // export / import; format-mediated  
    Mutation,      // direct client mutation via daemon; default  
}  
  
impl Thread {  
    /// Does this thread hold under this channel?  
    /// The load-bearing question.  
    pub fn holds_under(&self, channel: Channel) → Result<(), FrayOrSnap>;  
}
```

This is the v0.1.1 correction made structural. Load isn't a footnote on the check — it's an argument to it. Every enforcement question in the system compiles down to `holds_under(channel)` .

Type sketch is not final. Cody's Phase 0 scoping read on §4 is still open (bead `threads-986.5`).

Enforcement flow

From client to `ward.audit`.



Permit

All threads hold. Write proceeds.

DegradeToProposal

Frayed. Staged to `~/.coven/pending/`.
Principal notified.

Reject

Snapped or missing thread. Surface goes
read-only until repair.

The audit store

One daemon-owned store.

`coven-threads` ' event and audit log **extends** the existing `~/.coven/coven.sqlite3` — it does not stand up a parallel store.

⚠ Two audit stores = drift

Any sidecar file, separate DB, or "just for now" JSONL creates two sources of audit truth. When they disagree, neither one is authoritative — and by then it's already too late.

✓ Phase 0 decision

`ward.audit` is a **table inside** `coven.sqlite3`. Reachable through the existing socket. Daemon-owned. Non-negotiable.

Nova non-negotiable #3. WARD-C6 compaction ledger appends here. RFC-0001 §5.6 defines `ward_hash` as an audit-log field on this store.

Compatibility

RFC-0001 *wins* on conflict.

coven daemon

Imported as a crate. Zero socket-protocol changes in Phase 0/1/2. Wire format identical for clients.

Coven Cave

Consumes state via the daemon's HTTP API. New weave / thread / strand endpoints in Phase 4. No breaking changes before then.

⚠ Not `.af` -compatible

Letta's `.af` format strips `read_only` at export (source-verified 2026-07-14 against `letta-ai/letta/main/letta/serialize_schemas/pydantic_agent_schema.py`). Documented divergence, not accidental.

RFC-0001

Version pin v0.2.0+. This repo is a *conforming implementation* of §5. If we disagree with the RFC, this repo is wrong.

Grimoire · Ward Spec Brief §9

Normative reference for WARD-C1-C7. Inherited by citation; C7 landed there via bead `threads-986.12`.

Honest labeling

What Phase 0 *delivered*. And didn't.

Delivered ✓

- Repo scaffolded at `OpenCoven/coven-threads`
- Design doc v0.2 frozen (358 lines, 12 §)
- Vocabulary bound to concrete referents
- Type sketch — `Channel` , `Thread` , `Weave` , `Strand` , `PatternPredicate`
- Beads scaffolded (prefix `threads-`)
- Nova formal sign-off · RFC-0001 §5 round-trip verified
- WARD-C7 canonicalized in Grimoire §9

Not yet

- No enforcement code
- No Rust crate scaffold (Phase 1)
- Cody's §4 scoping read still open
- No daemon integration (Phase 2)
- No portability format (Phase 3)
- No Cave UX (Phase 4)
- Repo currently private

This is a design freeze, not a shipped layer. If we tell you it's more than that, correct us in Discord.

Two rules we're promoting

Coven, remember these.

✓ Rule one

Enforcement lives on the *authoritative* object.

Legibility lives on the *derived* one.

Predicates, not descriptors. Sources, not indexes. If you find yourself gating on a derived view because it's convenient, you're building the derived-index problem one layer up.

✓ Rule two

Check state *before* acting.

Not *after*.

The gate returns Permit / DegradeToProposal / Reject *before* the write hits disk. Post-hoc audit is a fallback, not a strategy.

What's next

The phase map.

Phase 0 · done

Design freeze. Beads scaffolded.
Vocabulary bound.

Phase 1 · Cody's lane

Rust crate `coven-threads-`
`core`. Types, tests, hash-
manifest, RFC-0001 §5
conformance suite.

Beads `threads-986.6 - .11`

Phase 2 · daemon

`validate()` call site inside the
socket handler. `ward.audit`
table. Proposal staging.

Phase 3 · portability

Familiar Portability Format v0.1.
Round-trip suite. C7 enforced
across serialization.

Phase 4 · Cave UX

Weave rail. Thread detail pane with tension state. Strand inspection. Proposal approval flow. Charm
voice/copy pass across all four surfaces.

Come build

The *weave holds* or it doesn't.

If you care about agentic memory that survives compaction, injection, serialization, and any writer who shouldn't have a pen — the design is frozen and the enforcement is where we go next.

Read

- `OpenCoven/coven-threads` · `specs/PHASE-0-DESIGN.md`
- `OpenCoven/familiar-contract` · RFC-0001 §5
- `OpenCoven/coven-grimoire` · Ward Layer Spec Brief §9

Contribute

- Beads open: `threads-986.5` Cody scoping read
- Discord: *#coven-threads* once we flip public
- Grimoire PR review: WARD-C7 canonicalization

The Rust daemon is the authority boundary. Every client is untrusted for enforcement purposes.

— `coven/docs/SAFETY-MODEL.md` · RFC-0001 §5.1 makes this a conformance requirement. `coven-threads` is the gate-shaped receiver behind that boundary.